

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approach

Introduction

The implementation phase focuses on converting the system design into a fully functional web application. The Smart Event Management and Ticket Booking System was developed using a modern full-stack architecture consisting of React.js for the frontend, Node.js with Express.js for the backend, and MongoDB Atlas as the cloud database. The application follows a modular approach where each functional component, such as user authentication, event management, booking management, and payment management, is developed independently and later integrated into a single system.

The frontend communicates with the backend through RESTful APIs, while MongoDB Atlas securely stores all application data. Version control was maintained using Git and GitHub, and the completed application was deployed using Vercel for the frontend and Railway for the backend.

5.2 Input and Output Design Implementation

Input Design

The system accepts user inputs through interactive web forms. Validation is performed at both the client side and server side to ensure data accuracy before processing.

The major input forms include:

- User Registration Form
- User Login Form
- Admin Login Form
- Event Creation Form
- Event Booking Form
- Payment Details Form

Validation includes:

- Required field validation
 - Email format validation
 - Password validation
 - Numeric value validation
 - Duplicate email prevention
 - Booking verification
-

Output Design

The system provides meaningful outputs to users after processing their requests.

Major outputs include:

- Registration Confirmation
- Login Authentication Status
- Event Listings
- Booking Confirmation
- Payment Confirmation
- User Dashboard
- Admin Dashboard
- Error and Validation Messages

The outputs are displayed through responsive Bootstrap-based user interfaces, ensuring compatibility across desktop and mobile devices.

5.3 Database Implementation

MongoDB Atlas was used as the cloud database for storing application data. Mongoose was used as the Object Data Modeling (ODM) library to establish communication between the backend and the database.

The database stores information related to:

- Users
- Events
- Bookings
- Payments

Each collection is represented using a Mongoose schema, ensuring consistency, validation, and efficient data retrieval.

The database connection is established securely using environment variables to protect sensitive credentials.

5.4 Table Structures

Although MongoDB is a NoSQL database, the application maintains structured collections equivalent to relational database tables.

User Collection

Field	Data Type	Description
_id	ObjectId	Unique User Identifier
name	String	User Name
email	String	Email Address
password	String	Encrypted Password
role	String	User/Admin Role
createdAt	Date	Account Creation Date

Event Collection

Field	Data Type	Description
_id	ObjectId	Event Identifier
title	String	Event Name

description	String	Event Description
category	String	Event Category
venue	String	Event Venue
eventDate	Date	Event Date
ticketPrice	Number	Ticket Price
availableSeats	Number	Number of Seats

Booking Collection

Field	Data Type	Description
_id	ObjectId	Booking Identifier
userId	ObjectId	User Reference
eventId	ObjectId	Event Reference
quantity	Number	Number of Tickets
totalAmount	Number	Total Booking Amount
bookingStatus	String	Booking Status
bookingDate	Date	Booking Date

Payment Collection

Field	Data Type	Description
_id	ObjectId	Payment Identifier
bookingId	ObjectId	Booking Reference
paymentAmount	Number	Amount Paid
paymentMethod	String	Payment Method
paymentStatus	String	Payment Status

paymentDate Date Transaction Date

5.5 Code Module

The application follows a modular architecture for better maintainability and scalability.

Frontend Modules

- Components
 - Pages
 - Layouts
 - Context API
 - Hooks
 - Services
 - Utilities
 - CSS Styling
-

Backend Modules

- Controllers
 - Models
 - Routes
 - Middleware
 - Configuration
 - Utility Functions
-

Major Functional Modules

- User Authentication Module
 - Event Management Module
 - Booking Management Module
 - Payment Management Module
 - Administrator Module
-

5.6 System Implementation

The complete system was implemented using the following technology stack.

Frontend

- React.js
 - Vite
 - Bootstrap 5
 - React Router DOM
 - Axios
-

Backend

- Node.js
 - Express.js
-

Database

- MongoDB Atlas
 - Mongoose
-

Authentication

- JSON Web Token (JWT)
 - bcrypt Password Encryption
-

Version Control

- Git
 - GitHub
-

Deployment

The application was successfully deployed using cloud platforms.

Frontend

- Vercel

Backend

- Railway

Database

- MongoDB Atlas

The frontend communicates with the backend through REST APIs, while the backend securely connects to MongoDB Atlas for data storage. Environment variables are used to manage sensitive configuration values such as database connection strings and authentication secrets.